

DevSecOps: Securing the CI/CD Pipeline

What DevSecOps Means

Security that **ships with the software**, not "security review" bolted at the end. The shift-left principle: find bugs where they're cheapest (in the developer's editor, not in production).

The Pipeline and Its Gates



"Shift left" = run the cheap safety checks as early as possible; the expensive ones (DAST, pen test) stay near the end.

The Tools by Stage

Stage	Tool type	Example
IDE/pre-commit	secrets scan	gitleaks, detect-secrets
Commit	SAST (static app scan)	semgrep, codeql
Build	SCA (dependency scan)	trivy, osv-scanner, Dependabot
Container	image scan	trivy, gype, scan on top of base image
Deploy	IaC scan	checkov, tfsec (M18-1)
Package	registry integrity	sbom signing (cosign)

The Secrets Problem (the #1 CI leak)

Hardcoded keys in code (the Google-breachers dream, real incidents everywhere)

Files like `.env`, `kubeconfig` committed by mistake

Fix: secret managers (GitHub Actions secrets / Vault / cloud key vault), rotation, and `gitleaks` / `detect-secrets` as a pre-commit hook.

```
gitleaks detect --source . --report-path leaks.json
# anything with a high entropy string / keyword = BLOCK build
```

The Supply-Chain Dimension (M08's SolarWinds-cousin)

Dependencies you pull (npm/pip/go) can be malicious or hijacked

Registry mirroring & pinning versions (no `latest`)

Signing (package provenance, cosign for images)

Software Bill of Materials (SBOM) – 'what's IN this artifact' as a machine-readable list – for recall + audit

The Pipeline Is Itself the Attack Surface

Compromised `CI secrets` = attacker ships your next release!

Supply-chain attacks on the CI runner, token theft

Guardrails: least-privilege CI tokens, pin the runner images, audit pipeline config, never build with prod secrets in env

The paradigm: every 'automated trust' is an attack surface. Automation accelerates humans – but it also accelerates *mistakes* unless gated. Gates are the name of the game.

[← Cloud Security Model & IAM](#)

Module complete ✓